Overlay + Persist

Complete guide: prepare the master SD card, test it, clone it to ~40 cabinets. Technicians only install and set the DIP switch.

WARNING - bench test first

- Not yet tested on a physical Raspberry Pi; only the scripts' logic was simulated. Do the full test (section 5) on the master and on ONE cloned SD before cloning all units.
- The standard Hardening package (A) is the lower-risk choice. Use this package only if the overlay is required.

1. How it works

With the overlay on, all writes go to RAM; the SD card is not written, so power cuts cannot corrupt it. A small root helper copies only the files that must survive to the real SD card (/media/root-ro), by unlocking it for a few milliseconds:

Item	Behaviour
config.json	Permanent (copied on every change)
conf.d/bridge.conf + /etc/hostname + /etc/netplan/*.yaml	Permanent, saved together, so the cabinet identity and its IP profile survive reboots
status.json, rfid1.json, rfid2.json, logs/	RAM only; revert to the image version at reboot (accepted)

Why netplan is included: the IP set by the DIP switch lives in a NetworkManager profile that is generated from /etc/netplan at every boot. Without persisting it, a reboot would restore the master's IP and ipservice.py could rewrite bridge.conf wrongly. The package also makes ipservice wait (After=dip.service + 30 s) and save the network profile before it reads the IP.

2. Package contents (new files only; no existing code is changed)

File	Purpose
overlay_persist/sitecustomize.py	Atomic writes; after a protected file is written under overlay, calls the helper.
overlay_persist/broker-persist	Root helper. Accepts only config.json, conf.d/bridge.conf, net. Rejects symlinks and paths outside the broker folder; validates JSON.
setup_overlay_persist.sh	Installs helper, restricted sudo rule, systemd drop-ins (send, ipservice). Does not enable the overlay.
check_overlay_persist.sh	Read-only status report: overlay, install, live-vs-SD comparison.
update_broker.sh	Updates broker code with overlay on, without touching data files.
remove_overlay_persist.sh	Uninstalls the package.

3. Who does what

Who	Does
You (preparer)	Prepare and test the master, create the SD image, write it to each cabinet SD in the workshop/before shipping.
Technician	Installs the unit, sets the DIP switch to the cabinet number, powers on, waits for one automatic reboot. No commands. See the one-page technician guide.

4. Prepare the master (overlay still OFF)

- Set the master's DIP switch to the master IP (e.g. 101) and boot. Check `findmnt -no FSTYPE /` prints **ext4**.
- Copy the **overlay_persist/** folder and the .sh files into /home/alice/broker/.

```
cd /home/alice/broker
chmod +x *.sh overlay_persist/broker-persist
sudo apt-get install -y rsync        # if missing
sudo ./setup_overlay_persist.sh
sudo reboot
```

Critical check BEFORE enabling the overlay - confirm that a DIP change is written to the permanent netplan file (this was assumed, never verified):

```
grep -H addresses /etc/netplan/*.yaml ; ls -l --time-style=full-iso /etc/netplan/
# change the DIP to another value (e.g. 105), wait for dip.py / reboot, then:
grep -H addresses /etc/netplan/*.yaml ; ls -l --time-style=full-iso /etc/netplan/
# the address must now be 192.168.137.105 and the mtime must be new
```

If the yaml does NOT change, stop: the package cannot keep the DIP IP across reboots on this system. Return the DIP to the master value.

If it changed: return the DIP to the master value, then enable the overlay and verify:

```
sudo raspi-config nonint do_overlayfs 0
sudo reboot
# after reboot:
sudo ./check_overlay_persist.sh      # all lines should be [ OK ]
```

If `/media/root-ro` does not exist, stop and disable the overlay (section 8).

5. Tests (all must pass)

#	Test	Expected
T1	Change a setting (config.json) on the web page; wait for the auto reboot	Setting still present
T2	Change the setting, cut power 1-2 s later; repeat 10-20 times	config.json always valid JSON (old or new value)
T3	Send scans, reboot	rfid reverts to image (accepted); broker works
T4	Measure SD writes (section 9)	Close to zero
T5	With overlay ON, change the DIP to another value; wait for the reboot; reboot again twice	IP, hostname and bridge.conf all match the DIP; no reboot loop; check script all [OK]
T6	Right after T5 changes, cut power instead of waiting	After power-up the unit is still consistent (same DIP IP/hostname/bridge.conf) or repeats the change once
T7	**Clone test:** image the master (section 6), write to a SECOND SD, set a different DIP (e.g. 103), boot	After one automatic reboot: IP .103, hostname matches, bridge.conf matches; stable across further reboots

Check the helper log: `journalctl -t broker-persist`. Finally restore the master to its original state (DIP = master IP, same files).

6. Create the image and clone

- On the master: `sudo poweroff`, wait until it is fully off, then take the SD card out. Never image a running card.
- Create the image on a PC (Raspberry Pi Imager / dd / Win32DiskImager) and write it to each cabinet SD; use cards of equal or larger size.
- Label each card with its cabinet number. Do not boot all clones on the same network at once: they share the master IP until the DIP is set.
- On the cabinet: the DIP switch is set once; dip.py changes the IP, ipservice.py rewrites bridge.conf and hostname, the helper stores them, and the unit reboots once. Later reboots keep everything.

Capacity limits in the provided code

- dip.py has 4 bits: only 15 addresses (.101-.115). ipservice.py maps .101-.126 to hostnames A-Z. Forty cabinets (.101-.140) exceed both. Confirm how the remaining cabinets are addressed before production; this package does not change that code.

7. Updating the broker later

```
# extract the new broker.zip to /home/alice/new_broker
sudo ./update_broker.sh /home/alice/new_broker
sudo reboot
```

config.json, status.json, rfid*.json, bridge.conf, logs/ and cloud/ are never overwritten. New config keys are not applied automatically. For many remote units, test the update on one unit first.

8. Disable the overlay / recovery

```
sudo raspi-config nonint do_overlayfs 1
sudo reboot
sudo ./remove_overlay_persist.sh      # optional, after boot
```

- Reboot loop: over SSH between two reboots run the first two lines above.
- No SSH: put the SD in a PC, edit **cmdline.txt** (bootfs) as a single line and remove the overlay entry (e.g. overlayroot=tmpfs).
- Keep the master image as backup; a dead remote unit is recovered by swapping the SD.

9. Risks and measuring benefit

Item	Level	Notes
Remount window	Low-Medium	A power cut during the few ms the SD is writable can hit the OS partition. Rare, not zero; happens only when config/bridge/network changes.
NM to netplan write-back	Unverified	Verified by the check in section 4. If it fails, do not use this package.
First boot of a clone	Low	Unit briefly has the master IP; do not boot several unset clones on one network.
Never tested on hardware	Unknown	Run T1-T7 first.

Measure SD writes (sectors) over 10 minutes of normal operation, with and without overlay:

```
awk '{print $7}' /sys/block/mmcblk0/stat
sleep 600
awk '{print $7}' /sys/block/mmcblk0/stat
```